

Introduction to Python

04 June 2026 17:04

Features of Python

- Python is a high level language. It is a free and open source language.
- It is an interpreted language, as Python programs are executed by an interpreter.
- Python programs are easy to understand as they have a clear defined syntax and relatively simple structures.
- Python is case sensitive. For example Number and number are not same in Python.
- Python is portable and platform independent means it can run on various operating systems and hardware platforms.
- Python has a rich library of predefined functions.
- Python is also helpful in web development. Many popular web services and applications are built using Python.
- Python uses indentation for blocks and nested blocks.

It was initially designed by Guido van Rossum in 1991 and developed by the Python Software Foundation. The language was conceived in the late 1980s, and its implementation began in December 1989 at Centrum Wiskunde & Informatica (CWI) in the Netherlands.

Working with Python

To write and run (execute) a program. We need to have a Python interpreter installed in our computer. Or we can use any online Python Interpreter.

```
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> 3 + 8
11
>>> print(4 + 5)
9
>>> print('Hello!')
Hello!
>>>
```

REPL → Loop → wait for the next command until the session ends.

Read → Accepts user input and parses it into Python expression.

Evaluate → Execute the parse code

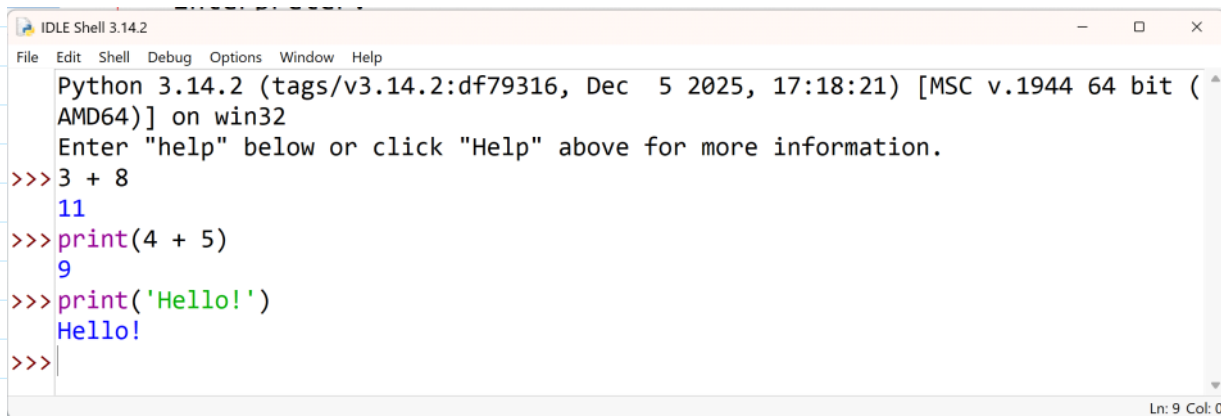
Print → Displays the result of execution.

Python interpreter or shell.

Execution Mode: → Interactive mode.
→ Script mode.

- **Interactive Mode:** To work in the interactive mode, we can simply type a Python statement on the > > > Prompt directly. As soon as we press enter, the

interpreter executes the statement and displays the results.



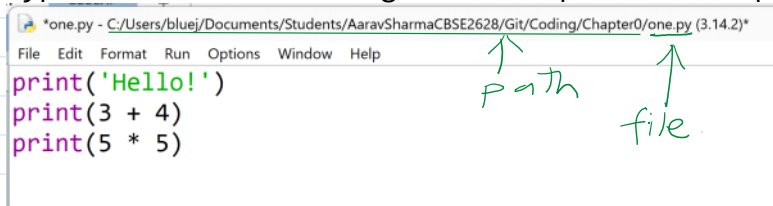
The screenshot shows the IDLE Shell 3.14.2 window. The title bar reads 'IDLE Shell 3.14.2'. The menu bar includes 'File', 'Edit', 'Shell', 'Debug', 'Options', 'Window', and 'Help'. The main text area displays the following content: 'Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32', 'Enter "help" below or click "Help" above for more information.', and a series of interactive commands and their outputs: '>>> 3 + 8' followed by '11', '>>> print(4 + 5)' followed by '9', and '>>> print('Hello!')' followed by 'Hello!'. The prompt '>>>' is shown at the end of the last line. The status bar at the bottom right indicates 'Ln: 9 Col: 0'.

Working in the interactive mode is convenient for testing a single line code for instant execution. But in the interactive mode, we cannot save the statements for future use and we have to retype the statements to run them again.

• Script Mode

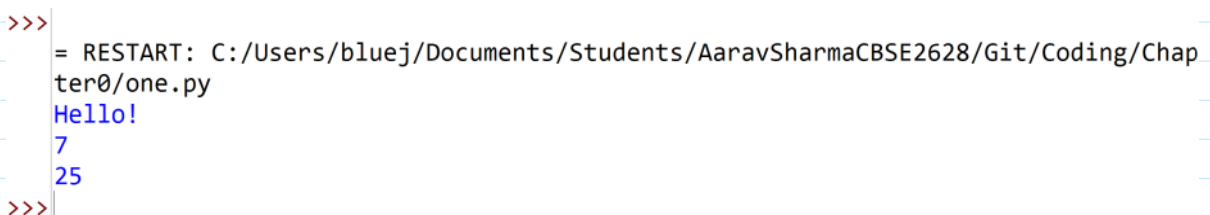
In the script mode, we can write a Python program in a file, save it and then use the interpreter to execute it. Python scripts are saved as files where filename has extension ".py". By default, the Python scripts are saved in the Python installation folder. To execute a script we can either:

- Type the file name along with the path at the prompt.



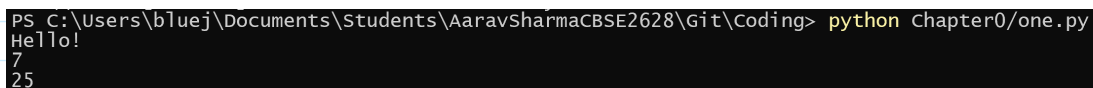
The screenshot shows a text editor window titled '*one.py - C:/Users/bluej/Documents/Students/AaravSharmaCBSE2628/Git/Coding/Chapter0/one.py (3.14.2)*'. The menu bar includes 'File', 'Edit', 'Format', 'Run', 'Options', 'Window', and 'Help'. The main text area contains three lines of Python code: 'print('Hello!')', 'print(3 + 4)', and 'print(5 * 5)'. Handwritten green arrows point from the words 'Path' and 'file' to the file path and filename in the title bar, respectively. The word 'Source code' is written in green below the code area.

Output:



The screenshot shows the IDLE Shell with the following output: '>>>' followed by '= RESTART: C:/Users/bluej/Documents/Students/AaravSharmaCBSE2628/Git/Coding/Chapter0/one.py', 'Hello!', '7', and '25'. The prompt '>>>' is shown at the end of the last line.

Or



The screenshot shows a command prompt window with the following text: 'PS C:\Users\bluej\Documents\Students\AaravSharmaCBSE2628\Git\Coding> python Chapter0/one.py', 'Hello!', '7', and '25'.

PYTHON Keywords

Keywords are reserved words. Each keyword has a specific meaning to Python interpreter and we can use a keyword in our program only for the purpose for which it has been defined. Tika as Python is case sensitive, keywords must be written exactly as given.

import keyword

```
print("Printing All Python keywords")
print("Python keywords are: ")
print(keyword.kwlist)
```

Output:

```
Printing All Python keywords
Python keywords are:
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break',
'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for',
'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not',
'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']
```

```
import keyword
```

```
print("Printing All Python keywords")
print("Python keywords are: ")
keyWordlist = keyword.kwlist
print(keyWordlist)
print("Number of keyword:",len(keyWordlist))
```

= RESTART:

```
C:/Users/bluej/Documents/Students/AaravSharmaCBSE2628/Git/Coding/Chapter0/keywo
rd.py
```

```
Printing All Python keywords
Python keywords are:
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break',
'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for',
'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not',
'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']
Number of keyword: 35
```

Identifiers or variables

In programming languages, identifiers are names used to identify a variable, function or other entities in Python program. The rules for naming an identifier in Python are as follows:

- The name should begin with an uppercase or a lowercase alphabet, or an underscore '_' sign. Thus an identifier cannot start with a digit.
- It can be of any length. (However, it is preferred to keep it short and meaningful.)
- It should not be a keyword or reserve word.
- We cannot use special symbols like !, @, #, \$, % etc. in identifiers.

Variables: A variable in a program is uniquely identified by a name (identifier). Variable in Python refers to an object - an item or element that is stored in the memory. Value of the variable can be string, numeric or any combination of alphanumeric characters. In Python we can use an assignment statement to create new variables and assign a specific values to them.

For example:

```
message = 'Keep Smiling'
print(message)
userNo = 101
print(f'User number is {userNo}')
```

Output:

```
Keep Smiling
User number is 101
```

For variable declaration is implicit in Python means variables are automatically declared and defined when they are assigned a value the first time. Variables must always be assigned values before they are used in expression as otherwise it will lead to an error in the program. So whenever a variable name occurs in an expression, the interpreter replaces it with the value of the particular variable.

← *Comment* ← Python interpreter will skip this line for interpretation.

```
# Write a Python program to find the area of a rectangle given that its
# length is 10 units and breadth is 20 units.
# main_code
length = int(input('Enter length: '))
breadth = int(input('Enter breadth: '))
area = length * breadth
print(f'area of rectangle: {area} sq.units')
```

Output:

```
Enter length: 23
Enter breadth: 17
area of rectangle: 391 sq.units
```

Everything is an object

Python treats every value or data item, whether numeric, string or other type. As an object in the sense that it can be assigned to some variable or can be passed to a function as an argument.

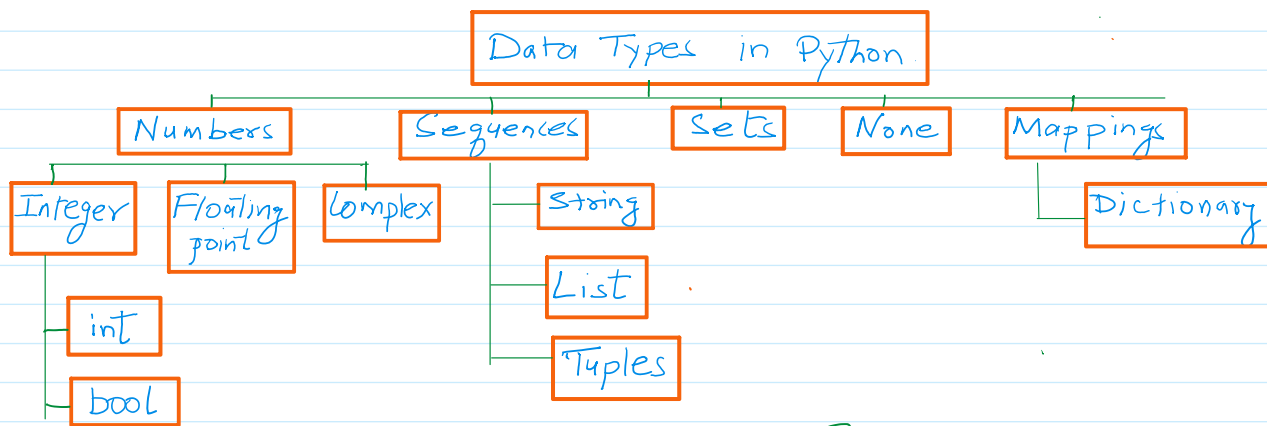
Every object in Python is assigned a unique identity (ID) which remains the same for the lifetime of that object. The function `id()` returns the identity of an object.

```
>>> name = 10
>>> type(name)
<class 'int'>
>>> id(name)
140711393785240
>>> name = 'Aarav'
>>> type(name)
<class 'str'>
>>> id(name)
2843222757744
>>> x = 10
>>> id(x)
140711393785240
>>> x = 11
>>> id(x)
140711393785272
```

For every distinct value a separate id is maintained by the Python automatically.

Data types

Every value belongs to a specific data type in Python. Data type identifies the type of data values a variable can hold and the operation that can be performed on that data. The following figure enlist the data type available in Python.



Different data types in Python.

$$\sqrt{-25} = \sqrt{(-1)(25)}$$

$\begin{matrix} 5 & \times & i \\ \hline 5 & i \end{matrix}$

Number

Number data type stores numerical values only. It further classified into three different types: bool, int, float and complex.

Numeric data types

Type/Class	Description	Examples
int	Integer numbers	-12, 0, -3, 15, 29, and many more
float	Real or floating point numbers	-2.05, 4.0, 14.23, many more
complex	complex number	<pre>>>> c = 5j >>> type(c) <class 'complex'> >>> d = 0j >>> type(d) <class 'complex'></pre>
bool	Boolean data type is a sub-type of integer. It can store two values either True or False.	<pre>>>> m = True >>> type(m) <class 'bool'> >>> n = False >>> type(n) <class 'bool'></pre>

Sequence

A Python sequence is an ordered collection of items, where each item is indexed by an integer. The three types of sequence data types available in Python are

strings, list and tuples.

A brief introduction to these data types is as follows:

String: String is a group of characters. These characters may be alphabets, digits, or special characters including spaces. String values are enclosed either in single quotation marks or in double quotation marks. The codes are not a part of strength. They are used to mark the beginning and end of the string for the interpreter. For example:

```
>>> name = "Aarav"
>>> name1 = 'Ajay'
>>> name
'Aarav'
>>> name1
'Ajay'
```

List: List is a sequence of items separated by commas and the items are enclosed in brackets [].

```
>>> li = []
>>> li1 = [5, 10.5, 34, -19, 'Hello', '20C', 200]
>>> li1
[5, 10.5, 34, -19, 'Hello', '20C', 200]
>>> li
[]
>>>
>>> li2 = [[1,2,3], [4,5,6], 100, 'Aarav']
>>> print(li2)
[[1, 2, 3], [4, 5, 6], 100, 'Aarav']
>>> li2[0]
[1, 2, 3]
>>> li2[0][0]
1
>>> li2[0][1]
2
```

Tuple

Tuple is a sequence of items separated by commas and items are enclosed in parentheses (). This is unlike list where values are enclosed in brackets []. Once created, we cannot change the tuple.

```
>>> tuple2 = (10, 20, 'Apple', [1,2,30], 'Hello', 3.5, -5j)
>>> print(tuple2)
(10, 20, 'Apple', [1, 2, 30], 'Hello', 3.5, (-0-5j))
>>> tuple2[2]
tuple2[2]
>>> tuple2[2]
'Apple'
>>> tuple2[3]
[1, 2, 30]
>>> tuple2[6]
(-0-5j)
>>> tuple2[3][2]
30
```

Set

Set is an unordered collection of items separated by commas, and the items are enclosed in curly brackets {}. A set is similar to a list except that it cannot have duplicate entries. Once created, elements of a set cannot be changed.

For example:

```
>>> set1 = {23, 3.14, 'New Delhi', (12, 14, 16)}
>>> print(set1)
{'New Delhi', 3.14, (12, 14, 16), 23}
>>> set2 = {1,2,1,3}
>>> set2
{1, 2, 3}
```

None: None is a special data type with a single value. It is used to signify the absence of value in a situation. Python supports no special operations and it is neither False nor 0.

```
>>> myvar = None
>>> type(myvar)
<class 'NoneType'>
>>> myvar
>>> print(myvar)
None
```

Mapping: Dictionary in Python holds data items in key - value pairs. Items in dictionaries are enclosed in curly brackets {}. Dictionaries permit faster access to data. Every key is separated from its value using a colon (:) sign. The key: value pairs of a dictionary can be accessed using the key. The keys are usually strings and their values can be any data type. In order to access any values in the dictionary we have to specify its key in brackets [].

```
>>> dict1 = {'Fruit':'Mango', 'Climate':'Hot and humid', 'Price(Kg)':60}
>>> dict1
{'Fruit': 'Mango', 'Climate': 'Hot and humid', 'Price(Kg)': 60}
>>> dict1['Climate']
'Hot and humid'
>>> dict1['Price(Kg)']
60
```

Mutable and Immutable Data Types

Sometimes we may require to change or update the values of a certain variables used in a program. However for a certain data type Python does not allow us to change the values of once a variable of that type has been created and assigned values.

Variables whose values can be changed after they are created and assigned are called mutable.

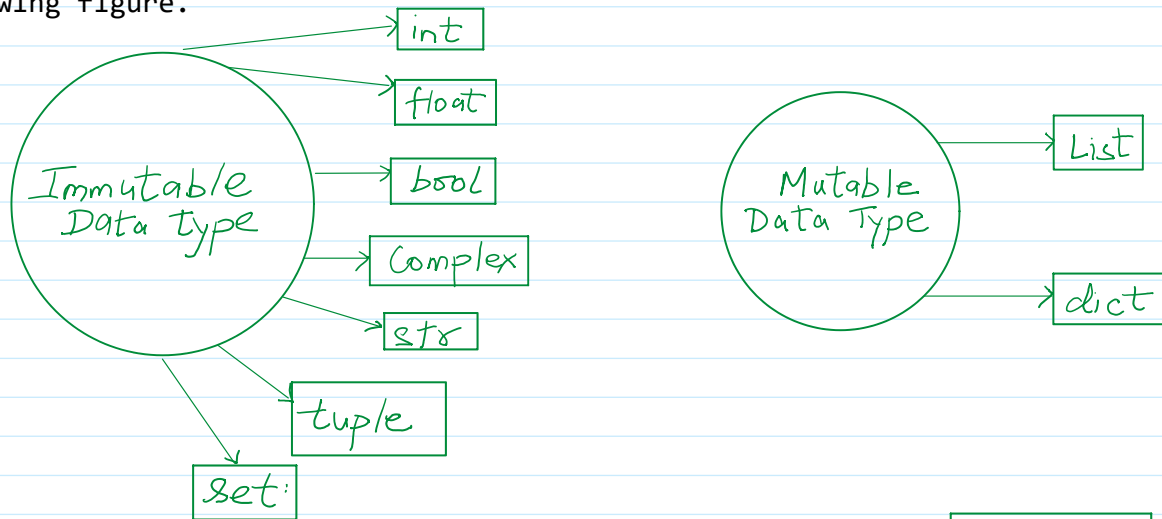
Variables whose values cannot be changed after they are created and assigned are called immutable.

When an attempt is made to update the value of an immutable variable, the old variable is destroyed and a new variable is created by the same name in memory.

Python data types can be classified into mutable and immutable as shown in the following figure.

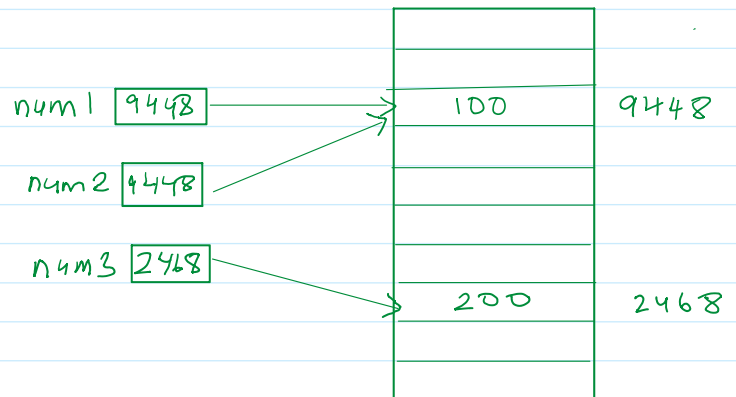


Python data types can be classified into mutable and immutable as shown in the following figure.



```

num1 = 100
id(num1)
140709404639448
num2 = 100
id(num2)
140709404639448
num3 = 200
id(num3)
140709404642648
  
```



Variables with different values have different identifiers.

Deciding usage of Python Data types

It is preferred to use list when we need a simple iterable collection of data that may go for frequent modifications. For example, if we store the names of students of a class in a list, then it is easy to update the list when some new students join or some leave the course. Pupils are used when we do not need any change in data. For example, names of months in a year. When we need uniqueness of elements and to avoid duplicity, it is preferable to use sets, for example lists of artifacts in a museum. If our data is being constantly modified or we need a fast lookup based on a custom key or we need a logical association between key:value pair. It is advisable to use dictionary. A mobile phone book is a good application of dictionary.

Operators

An operator is used to perform specific mathematical or logical operation on values. The values that the operator work on are called operands. For example, in the expression $10 + \text{NUM}$, The value 10 and the variable NUM are operands and the '+' is an operator. Python supports several kinds of operators whose categorization is briefly explained below:

Arithmetic Operators

Python supports arithmetic operators that are used to perform the four basic arithmetic operations, as well as modular division, floor division, and exponentiation.

Arithmetic Operators in Python

Operator	Operation	Description	Example
+	Addition	Adds the two numeric values on either side of the operator.	num1 100 num2 100 num3 200 r = num1 + num3 r 300
	Concatenation	This operator can also be used to concatenate two strings on either side of the operator.	fn = 'Aarav' ln = 'Sharma' spc = ' ' fullNm = fn + spc + ln fn 'Aarav' ln 'Sharma' fullNm 'Aarav Sharma'
-	Subtraction	Subtracts the operand on the right from the operand on the left.	r 300 num2 100 diff = r - num2 diff 200
*	Multiplication	Multiplies the two values on both side of the operator.	x = 8 y = 9 p = x * y p 72
		Repeat the item on the left of the operator if first operand is a string and 2nd operand is an integer value.	'p' * 20 'pppppppppppppppppppppp' fn 'Aarav' fn * 5 'AaravAaravAaravAaravAarav'
/	Division	Divides the operand on the left by the operand on the right and returns the quotient.	x 8 y 9 q = x / y q 0.8888888888888888
%	Modulus	Divides the operand on the left by the operand on the right, and returns the	x 8 y

		remainder.	9 r = x % y r 8
//	Floor division	Divides the operand on the left by the operand on the right and returns the quotient by removing the decimal part. It is sometimes also called integer division.	p 72 q = p // 10 q 7
**	Exponent	Performs exponential (power) Calculation on operands. That is. Raise the operand on the left to the power of the operand on the right.	x 8 p = x ** 2 p 64 rt = p ** 0.5 rt 8.0 cbirt = rt ** 0.3 cbirt 1.8660659830736148 cbirt = rt ** (1/3) cbirt 2.0

Relational Operators

Relational operator compares the values of the operands on its either side and determines the relationship among them. Assume the Python variables num1 = 10, num2 = 0, num3 = 10, str1 = "Good", str2 = "Afternoon" for the following examples:

Relational operators in Python

Operator	Operation	Description	Example
==	Equals to	If the value of two operands are equal, then the condition is true, otherwise it is false.	num1 == num2 False str1 == str2 False
!=	Not equal to	If values of two operands are not equal then condition is true, otherwise it is false.	num1 != num2 True str1 != str2 True
>	Greater than	If the value of the left side operand is greater than the value of the right side operand, then condition is true, otherwise it is false.	num1 > num2 True str1 > str2 True
<	Less than	Anyway, if the value of the left side operand is less than the value of the right side operand, then the condition is true, otherwise it is false.	num1 < num2 False str2 < str1 True

<code>>=</code>	Greater than or equal to	If the value of the left side operand is greater than or equal to the value of the right side operand, then condition is true. Otherwise it is false.	<pre>num1 >= num3 True str2 >= str1 False str1 >= str2 True</pre>
<code><=</code>	Less than or equal to.	If the value of the left operand is less than or equal to the value of the right operand, then it is true, otherwise it is false.	<pre>num1 <= num3 True str2 <= str1 True str1 <= str2 False</pre>

Assignment Operators

Assignment operator assigns or changes the value of a variable on its left.

Assignment operators in Python

Operator	Description	Example
<code>=</code>	Assigns values from right side operand to the left side operand.	<pre>num1 = 10 num2 = 0 str1 = "Good" str2 = "Afternoon"</pre>
<code>+=</code>	It adds the value of right side operand to the left side operand and assigns the result to the left side operand. Note: <code>x += y</code> is same as <code>x = x + y</code>	
<code>-=</code>		
<code>*=</code>		
<code>/=</code>		
<code>%=</code>		
<code>//=</code>		
<code>**=</code>		

Logical Operators

There are three logical operators supported by Python. These are operators (and, or, not) are to be written in lower case only. The logical operators. Evaluate to either true or false based on the logical operands on either side. Every value is logically either true or false. By default all values are True except None, False, 0, Empty collection: "", (), [], {} And few other special values.

Logical operators in Python

Operator	Operation	Description	Example
<code>and</code>	Logical AND	If both the operands are true then condition becomes true.	<pre>True and True True True and False False False and True False False and False</pre>

			False
or	Logical OR	If any of the two operands are true, then conditions become true.	True or True True True or False True False or True True False or False False
not	Logical NOT	Used to reverse the logical state of its operation operand.	not True False not False True

Identity Operators

Identity operators are used to determine whether the value of a variable is of a certain type or not. Identity operators can also be used to determine whether two variables are referring to the same object or not. There are two identity operators.

Operator	Description	Example
is	Evaluates True if the variables on either side of the operator points towards the same memory location, and False otherwise. var1 is var2 results to True if id(var1) is equal to var2	num1 = 5 num2 = num1 id(num1) 140704793261304 id(num2) 140704793261304 num1 is num2 True num3 = 6 num1 is num3 False id(num3) 140704793261336
is not	Evaluates to false if the variables on the either side of the operator points to the same memory location and true otherwise. Var1 is not var2 results to True if id(var1) is not equal to id(var2).	id(num1) 140704793261304 id(num2) 140704793261304 id(num3) 140704793261336 num1 is not num3 True num1 is not num2 False

Membership Operators

Membership operators are used to check if a value is a member of a given sequence or not.

Membership operators in Python

Operator	Description	Example
in	Returns true if the variable or value is found in the specified sequence, and falls otherwise.	<pre>li = [20, 30, 10, 70, 40, 60] 10 in li True 25 in li False</pre>
not in	The returns true if the variable or value is not found in the specified sequence, and false otherwise.	<pre>li [20, 30, 10, 70, 40, 60] 12 not in li True 70 not in li False</pre>

Expressions: An Expression is defined as a combination of constants, variables and operators. An expression always evaluates to a value. A value or a standalone variable is also considered as an expression, but a standalone operator is not an expression.

Precedence of Operators

Evaluation of the expression is based on precedence of operators. When an expression contains different kinds of operators, precedence determines which operators should be applied first. Higher precedence operator is evaluated before the lower precedence operator. Most of the operators studied till now are binary operators. Binary operators are the operators with two operands. The unary operator needs only one operand, and they have higher precedence than the binary operators. The minus as well as plus operators can act as both unary and binary operators. But **not** is a unary logical operator.

Precedence of all operators in Python

Here's the **complete precedence table of Python operators** (from **highest to lowest precedence**) along with their **associativity**.

This applies to **Python 3.11+** and is valid for Python 3.x in general.

Python Operator Precedence Table

Precedence (High → Low)	Operator(s)	Description	Associativity
1	()	Parentheses (grouping)	Left-to-Right
	f(args...)	Function call	Left-to-Right
	x[index]	Indexing / slicing	Left-to-Right
	x.attr	Attribute reference	Left-to-Right
	f"{expr}"	Formatted string literal	Left-to-Right
2	**	Exponentiation	Right-to-Left
3	+x, -x, ~x	Unary plus, unary minus, bitwise NOT	Right-to-Left
4	*, /, //, %	Multiplication, division, floor division, modulus	Left-to-Right

5	+, -	Addition, subtraction	Left-to-Right
6	<<, >>	Bitwise shift left, shift right	Left-to-Right
7	&	Bitwise AND	Left-to-Right
8	^	Bitwise XOR	Left-to-Right
9		Bitwise OR	Left-to-Right
10	in, not in, is, is not, <, <=, >, >=, !=, ==	Comparisons, membership, identity	Left-to-Right
11	not	Logical NOT	Right-to-Left
12	and	Logical AND	Left-to-Right
13	or	Logical OR	Left-to-Right
14	if ... else ...	Conditional expression	Right-to-Left
15	lambda	Lambda expression	Left-to-Right
16	:=	Assignment expression (walrus operator)	Right-to-Left
17	=, +=, -=, *=, /=, //=, %=, ^=, `>>=`, `<<=`, =, **=, &=, `	Assignment operators	Right-to-Left
18	yield, yield from	Yield expressions	Left-to-Right
19	,	Tuple, list, dict, set separators	Left-to-Right

Key Notes

- **Parentheses** can always be used to override precedence.
- **Exponentiation (**) is right-associative:**
 $2 ** 3 ** 2 \rightarrow 2 ** (3 ** 2) \rightarrow 2 ** 9 = 512$
- **Comparisons** can be chained:
 $3 < x < 10$ is equivalent to $(3 < x)$ and $(x < 10)$
- **Assignment expressions (:=)** have lower precedence than most operators but higher than assignment =.
- **Logical operators** (and, or, not) have lower precedence than comparison operators.

"""

Program to print the Python Operator Precedence Table

"""

```
def print_precedence_table():
    precedence = [
        (1, "Parentheses / Function Call / Subscription / Slicing / Attribute",
        "(a+b), f(x), a[0], a[1:5], obj.attr"),
```

```

    (2, "Await", "await x"),
    (3, "Exponentiation", "**"),
    (4, "Unary Operators", "+x, -x, ~x"),
    (5, "Multiplication, Matrix Multiplication,\n Division, Floor Division,
Modulus", "*", "@, /, //, %"),
    (6, "Addition, Subtraction", "+, -"),
    (7, "Bitwise Shift", "<<, >>"),
    (8, "Bitwise AND", "&"),
    (9, "Bitwise XOR", "^"),
    (10, "Bitwise OR", "|"),
    (11, "Comparison, Membership, Identity", "<, <=, >, >=, !=, ==, in, not
in, is, is not"),
    (12, "Logical NOT", "not"),
    (13, "Logical AND", "and"),
    (14, "Logical OR", "or"),
    (15, "Conditional Expression", "x if condition else y"),
    (16, "Lambda Expression", "lambda"),
    (17, "Assignment Expression", ":=")
]
print("PYTHON OPERATOR PRECEDENCE TABLE")
print("=" * 100)
print(f"{'Level':<6} {'Operator Category':<45} {'Operators / Example'}")
print("-" * 100)
for level, category, operators in precedence:
    print(f"{level:<6} {category:<45} {operators}")
print("-" * 100)
print("Note: Level 1 has the highest precedence, while Level 17 has the
lowest.")
if __name__ == "__main__":
    print_precedence_table()

```

Output:

PYTHON OPERATOR PRECEDENCE TABLE

Level	Operator Category	Operators / Example
1	Parentheses / Function Call / Subscription / Slicing / Attribute	(a+b), f(x), a[0], a[1:5], obj.attr
2	Await	await x
3	Exponentiation	**
4	Unary Operators	+x, -x, ~x
5	Multiplication, Matrix Multiplication, Division, Floor Division, Modulus	*, @, /, //, %
6	Addition, Subtraction	+, -
7	Bitwise Shift	<<, >>
8	Bitwise AND	&
9	Bitwise XOR	^
10	Bitwise OR	
11	Comparison, Membership, Identity	<, <=, >, >=, !=, ==, in, not in, is, is not
12	Logical NOT	not
13	Logical AND	and

14	Logical OR	or
15	Conditional Expression	x if condition else y
16	Lambda Expression	lambda
17	Assignment Expression	:=

 Note: Level 1 has the highest precedence, while Level 17 has the lowest.

Input and Output

The input () function prompts the user to enter data. It accepts all user input as a string. The user may enter a number or a string but the input function. Treats them as a string only. The syntax for input () is:

```
input("prompt")
```

Prompt is a string we may like to display on the screen prior to taking the input and it is optional. When prompt is specified first it is displayed on the screen, after which the user can enter data. The input () takes exactly what is typed from the keyboard, converts it into a string and assign it to the variable on the left hand side of assignment operator. (=). So Frankie entering data for the input function is terminated by pressing the enter key.

```
lname = input('Enter your last name: ')
```

```
Enter your last name: Sharma
```

```
age = int(input('Enter your age: '))
```

```
Enter your age: 16
```

```
type(age)
```

```
<class 'int'>
```

```
type(fname)
```

```
<class 'str'>
```

```
type(lname)
```

```
<class 'str'>
```

Output: Python uses print () function to output data to standard output device, the screen. The function print () evaluates the expression before displaying it on the screen. The print () Outputs a complete line and then moves to the next line for subsequent output.

Syntax:

```
print(value[, ...,] sep=' ', end='\n')
```

- **sep:** The optional parameter Sep is a separator between the output values. We can use a character, integer or a string as a separator. The default separator is a space.
- **end:** This is also optional and it allows us to specify any string to be appended after the last value. Default is a new line('\n').

Type Conversion

Explicit Conversion: Explicit conversion, also called typecasting, happens when data type conversion takes place because the programmer forced it in the program. The general form of an explicit type data type conversion is:

```
New_data_type(expression)
```

With explicit type conversion, there is a risk of loss of information since we are forcing an expression to be of a specific type.

For example:

Here fractional part 0.67 is lost.

For example:

not defined $\rightarrow -1! = 0! = 1! = 1$ not defined

$1! = 2!/2 = 1! = 1$

$2! = 3!/2 = 2!$

$3! = 4!/4$

$4! = 5!/5 = 4!$

$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$

$\frac{4 \times 3 \times 2 \times 1}{4} = 3!$

Debugging: Key A programmer can make mistakes while writing a program and hence the program may not execute or may generate wrong output. The process of identifying and removing such mistakes, also known as bugs or errors, from a program, is called debugging. Errors occurring in the program can be categorized as:

-
- unit circle concept.
- $\pi/4$
- π 2π
- $- +$ $+ +$
- $- -$ $+ -$